

1 Write a Python program using NumPy to create arrays and perform operations.

Aim

To write a Python program using NumPy that creates arrays and performs basic operations such as addition, subtraction, multiplication, division, and statistical functions.

Algorithm

1. Import the NumPy library.
2. Create arrays using .
3. Perform element-wise operations: addition, subtraction, multiplication, division.
4. Apply NumPy functions like , , .
5. Display results.

CODE

```
import numpy as np
```

```
# Step 1: Create arrays
```

```
arr1 = np.array([10, 20, 30, 40])
```

```
arr2 = np.array([1, 2, 3, 4])
```

```
print("Array 1:", arr1)
```

```
print("Array 2:", arr2)
```

```
# Step 2: Perform operations
```

```
print("\nAddition:", arr1 + arr2)
```

```
print("Subtraction:", arr1 - arr2)
```

```
print("Multiplication:", arr1 * arr2)
```

```
print("Division:", arr1 / arr2)
```

Step 3: Statistical operations

```
print("\nMean of Array 1:", np.mean(arr1))
```

```
print("Max of Array 2:", np.max(arr2))
```

```
print("Min of Array 2:", np.min(arr2))
```

OUTPUT

Array 1: [10 20 30 40]

Array 2: [1 2 3 4]

Addition: [11 22 33 44]

Subtraction: [9 18 27 36]

Multiplication: [10 40 90 160]

Division: [10. 10. 10. 10.]

Mean of Array 1: 25.0

Max of Array 2: 4

Min of Array 2: 1

Conclusion

- NumPy allows easy array creation and vectorized operations without explicit loops.
- Mathematical and statistical functions (, ,) are built-in and efficient.
- This program demonstrates how NumPy simplifies numerical computing in Python

2 Write a program using Pandas to create a DataFrame and display basic statistics.

Aim

To write a Python program using Pandas that creates a DataFrame and displays basic statistics such as mean, median, maximum, minimum, and standard deviation.

Algorithm

1. Import the Pandas library.
2. Create a DataFrame using with sample data.
3. Display the DataFrame.
4. Use built-in Pandas functions (, , ,) to compute statistics.
5. Print the results.

CODE

```
import pandas as pd
```

```
# Step 1: Create a DataFrame
```

```
data = {  
    'Name': ['Yogin', 'Alex', 'Priya', 'Sam'],  
    'Age': [25, 30, 22, 28],  
    'Score': [85, 90, 78, 88]  
}
```

```
df = pd.DataFrame(data)
```

```
# Step 2: Display the DataFrame
```

```
print("DataFrame:\n", df)
```

```
# Step 3: Display basic statistics
```

```
print("\nBasic Statistics:")
```

```
print("Mean Age:", df['Age'].mean())
```

```
print("Median Age:", df['Age'].median())
```

```
print("Max Score:", df['Score'].max())
print("Min Score:", df['Score'].min())
print("Standard Deviation of Score:", df['Score'].std())
```

Step 4: Use describe() for summary

```
print("\nSummary using describe():\n", df.describe())
```

OUTPUT

DataFrame:

	Name	Age	Score
0	Yogin	25	85
1	Alex	30	90
2	Priya	22	78
3	Sam	28	88

Basic Statistics:

Mean Age: 26.25

Median Age: 26.5

Max Score: 90

Min Score: 78

Standard Deviation of Score: 5.0

Summary using describe():

	Age	Score
count	4.000000	4.000000
mean	26.250000	85.250000
std	3.304038	5.058324
min	22.000000	78.000000
25%	24.250000	83.250000

50% 26.500000 86.500000

75% 28.500000 88.500000

max 30.000000 90.000000

Conclusion

- Pandas makes it easy to create and manage tabular data using DataFrames.
- Built-in functions like `mean`, `std`, `min`, `max`, and `describe` simplify statistical analysis.
- The `describe` method provides a quick summary of numerical columns.